

UNIT-2

CHAPTER 1- Process Synchronization

Syllabus:

Process Synchronization: The Critical Section Problem, Solution for Critical Section Problem, Bakery Algorithm, Semaphores-Meaning, Types of Semaphores, Synchronization Problems- Bounded Buffer Problem, Readers-Writers Problem.

PROCESS SYNCHRONIZATION

Definition:

Process synchronization is the task of coordinating the execution of processes in such a way that no two processes can access the same shared data and resources.

It aims to resolve the problem of race conditions and other synchronization issues in a concurrent system.

- Independent Process: The execution of one process does not affect the execution of other processes.
- Cooperative Process: A process that can affect or be affected by other processes executing in the system



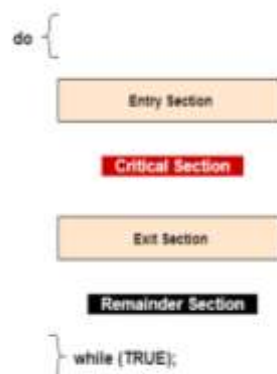
Process synchronization problem arises in the case of Cooperative processes also because resources are shared in Cooperative processes.

The main objective of process synchronization is to ensure that multiple processes access shared resources without interfering with each other and to prevent the possibility of inconsistent data due to concurrent access.

CRITICAL SECTION PROBLEM

When more than one processes try to access the same code segment that segment is known as the critical section.

Only one process can execute in its critical section at a time. All the other processes have to wait to execute in their critical sections.



The entry section handles the entry into the critical section. It acquires the resources needed for execution by the process. The exit section handles the exit from the critical section. It releases the resources and also informs the other processes that the critical section is free.

SOLUTION FOR CRITICAL SECTION PROBLEM

Critical section problem must satisfy three requirements:

- **Mutual Exclusion:** If a process is executing in its critical section, then no other process is allowed to execute in the critical section.
- **Progress:** If no process is executing in the critical section and other processes are waiting outside the critical section, then only those processes that are not executing in their remainder section can participate in deciding which will enter the critical section next, and the selection can not be postponed indefinitely.
- **Bounded Waiting:** A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

BAKERY ALGORITHM

The Bakery Algorithm is a simple process synchronization algorithm which is used for preventing the problem of race conditions in critical sections of the program or in an operating system.

The bakery algorithm is a mutual exclusion algorithm. Hence, it allows multiple processes to access the critical section of the program in a right manner.

This algorithm operates by giving a unique number to each process that requests for accessing to the critical section.

The bakery algorithm is based on the first come first serve property. Therefore, the process with the smallest number is given priority to access the critical section first. In case when two or more processes assigned the same number, then the process with lowest process ID is given priority.

In this algorithm, two arrays are maintained, one array is of flags to indicate that a process is currently being requesting to enter to the critical section, and another array is of numbers that indicate the order of processes in which they have requested to enter the critical section.

When a process requests to access the critical section, it first sets its flag to true and then selects the largest number in the array and increases it by one to obtain its own unique number. This process then waits to get access to the critical section.

When the process has entered the critical section, its flag is set to false which indicates that this process is no longer need to access the critical section. Hence, the process releases its number so that the other processes can use it.

Fundamentally, the bakery algorithm in process synchronization ensures that no two processes can access the critical section simultaneously. In this algorithm, a process may have to wait for indefinitely long time if the assigned number is too high.

Example:

- Process P1 takes ticket 1.
- Process P2 takes ticket 2.
- Process P3 takes ticket 3.
- P1 enters the critical section first, then P2, and finally P3.

The algorithm ensures no two processes access the critical section simultaneously and does so in an orderly, fair manner.

How it works?

1. Ticket Assignment:
Each process that wants to enter the critical section takes a unique "ticket" number, which is higher than any other process's ticket number.
2. Waiting for Turn:
Processes wait for their turn based on the ticket numbers. The process with the smallest ticket number enters the critical section first. If two processes have the same ticket, the one with the lower process ID gets priority.
3. Entering Critical Section:
A process enters the critical section only when no other process with a smaller or equal ticket number is executing.
4. Exiting Critical Section:
Once a process finishes its work in the critical section, it exits, allowing the next process with the smallest ticket to enter.

SEMAPHORES

The various hardware-based solutions to the critical-section problem are complicated for application programmers to use. To overcome this difficulty, we can use a synchronization tool called a semaphore.

- The Semaphore is just a normal integer.
- The Semaphore cannot be negative.
- The least value for a Semaphore is zero (0).
- The Maximum value of a Semaphore can be anything.
- The Semaphores usually have two operations.
- The two operations have the capability to decide the values of the semaphores.

The two Semaphore Operations are:

1. Wait ()
2. Signal ()

1. The Wait Operation is used for deciding the condition for the process to enter the critical state or wait for execution of process.

Here, if the Semaphore value is greater than zero or positive then the Process can enter the Critical Section Area.

If the Semaphore value is equal to zero then the Process has to wait for the Process to exit the Critical Section Area.

```
wait(S)
{
while (S<= 0) ;           // no-operation
S--;
}
```

This function is only present until the process enters the critical state. If the Processes enters the critical state, then the Wait Operation has no job to do.

2.The Signal Semaphore Operation is used to update the value of Semaphore. The Semaphore value is updated when the new processes are ready to enter the Critical Section.

```
Signal(S)
{
    S++;
}
```

We know that the semaphore value is decreased by one in the wait operation when the process left the critical state. So, to counter balance the decreased number 1 we use signal operation which increments the semaphore value. This induces the critical section to receive more and more processes into it.

TYPES OF SEMAPHORES:

Binary Semaphores:

- They can only be either 0 or 1.
- They are also known as mutex locks, as the locks can provide mutual exclusion.
- All the processes can share the same mutex semaphore that is initialized to 1.
- Then, a process has to wait until the lock becomes 0.
- Then, the process can make the mutex semaphore 1 and start its critical section.
- When it completes its critical section, it can reset the value of the mutex semaphore to 0 and some other process can enter its critical section.

Counting Semaphores:

- They can have any value and are not restricted to a certain domain.
- They can be used to control access to a resource that has a limitation on the number of simultaneous accesses.
- The semaphore can be initialized to the number of instances of the resource.
- Whenever a process wants to use that resource, it checks if the number of remaining instances is more than zero, i.e., the process has an instance available.
- Then, the process can enter its critical section thereby decreasing the value of the counting semaphore by 1.
- After the process is over with the use of the instance of the resource, it can leave the critical section thereby adding 1 to the number of available instances of the resource.

Advantages of Process Synchronization

- Ensures data consistency and integrity
- Avoids race conditions
- Prevents inconsistent data due to concurrent access

Disadvantages of Process Synchronization

- Increases the complexity of the system
- Can cause deadlocks if not implemented properly.

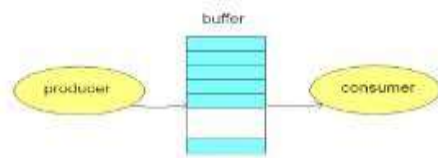
SYNCHRONIZATION PROBLEMS

Bounded Buffer Problem

The bounded-buffer problem (also known as the producer—consumer problem)

- It is a classic example of a multi-process synchronization problem.
- The problem describes two processes, the producer and the consumer, who share a common, fixed-size buffer used as a queue.
- The producer's job is to generate data, put it into the buffer, and start again. At the same time, the consumer is consuming the data (i.e., removing it from the buffer), one piece at a time.

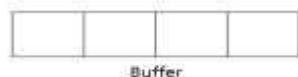
- The problem is to make sure that the producer won't try to add data into the buffer if it's full and that the consumer won't try to remove data from an empty buffer.



- The solution for the producer is to either go to sleep or discard data if the buffer is full.
- The next time the consumer removes an item from the buffer, it notifies the producer, who starts to fill the buffer again. In the same way, the consumer can go to sleep if it finds the buffer to be empty.
- The next time the producer puts data into the buffer, it wakes up the sleeping consumer. The solution can be reached by means of inter-process communication, typically using semaphores. An inadequate solution could result in a deadlock where both processes are waiting to be awakened. The problem can also be generalized to have multiple producers and consumers.

Producer

```
Count!=Buffersize
Buffer=product
Count=count+1
```



Consumer



Consumer

```
Count!=0
Consumer=product
Count=count-1
```

Readers - Writers Problem

A database is to be shared among several concurrent processes. Some of these processes may want only to read the database, whereas others may want to update (that is, to read and write) the database.

- In Readers - Writers Problems there are two Actors for the problem. They are Reader and Writer.
- The Reader - Writer Problem tries to access or change the value of the Shared Variable.
- Due, to this parallel execution of accessing and changing operation data faults take place.
- Due, to these expected outputs are not the actual outputs of the problem.

A solution to either problem may result in starvation. In the first case, writers may starve; in the second case, readers may starve

OS, we call this situation the readers-writer, problem

Problem parameters: that

- One set of data is shared among a number of processes
- Once a writer is ready, it performs its write. Only one writer may write at a time
- If a process is writing, no other process can read it
- If at least one reader is reading, no other process can write
- Readers may not write and only read

The first readers-writers problem, requires that no reader will be kept waiting unless a writer has already obtained permission to use the shared object. In other words, no reader should wait for other readers to finish simply because a writer is waiting.

The second readers-writers problem requires that, once a writer is ready, that writer performs its write as soon as possible. In other words, if a writer is waiting to access the object, no new readers may start reading.

Unit 2

Chapter 2 - DEADLOCKS

Syllabus:

Deadlocks: Deadlock Characterization, Methods for Handling Deadlocks, Deadlock Prevention, Deadlock Avoidance, Deadlock Detection, Recovery from Deadlock.

DEADLOCKS:

A deadlock is a situation in a system where two or more processes are unable to proceed because each is waiting for a resource that is held by another process. This leads to an indefinite waiting state where no process can make progress.

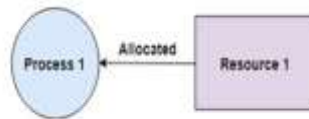
DEADLOCK CHARACTERIZATION

In deadlock, processes never finish executing and system resources are tied up, preventing other jobs from ever starting.

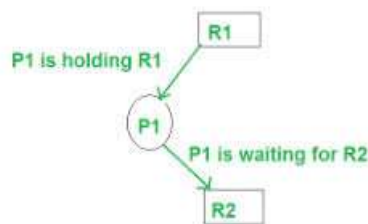
Necessary Conditions for deadlock

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

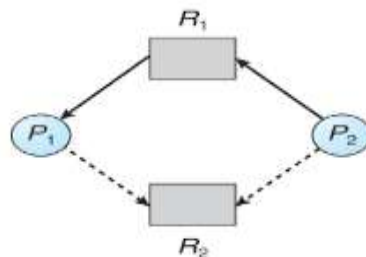
- Mutual exclusion: At least one resource must be held in a non-sharable mode; that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.



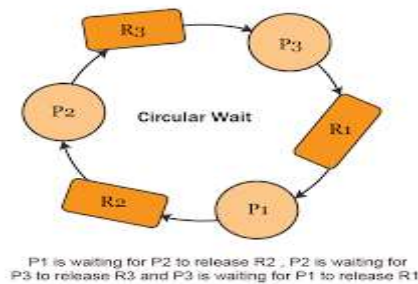
- Hold and wait: There must exist a process that is holding at least one resource and is waiting to acquire additional resources that are currently being held by other processes.



- No preemption: Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process, has completed its task.



- Circular wait: There must exist a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .



METHOD FOR HANDLING DEADLOCK

There are three different methods for dealing with the deadlock problem:

- We can use a protocol to ensure that the system will never enter a deadlock state.
- We can allow the system to enter a deadlock state and then recover.
- We can ignore the problem all together, and pretend that deadlocks never occur in the system.

Handling Methods:

1. Deadlock Prevention
2. Deadlock Avoidance
3. Deadlock Detection
4. Recovery from Deadlock

1) DEADLOCK PREVENTION

For a deadlock to occur, each of the four necessary-conditions must hold. By ensuring that at least on one these conditions cannot hold, we can prevent the occurrence of a deadlock.

○ Mutual Exclusion

Only one process can use a non-sharable resource at a time. For example, a printer cannot be used by multiple processes simultaneously.

○ Hold and Wait

A process holds one resource while waiting to acquire more. This can lead to deadlock if not controlled properly.

○ No Preemption

Resources cannot be forcibly taken from a process; they must be released voluntarily, which can cause delays in allocation.

○ Circular Wait

Processes wait on each other in a circular chain. This can be prevented by assigning a fixed order for resource requests.

2) DEADLOCK AVOIDANCE

Deadlock avoidance ensures that the system never enters an unsafe state by checking each request before granting it. It avoids circular wait by making decisions based on the current resource allocation.

1. Safe State

A system is in a safe state if all processes can complete without leading to deadlock. A safe sequence is an order in which processes can finish using available and released resources.

If no safe sequence exists, the system is in an **unsafe state**, which may lead to deadlock.

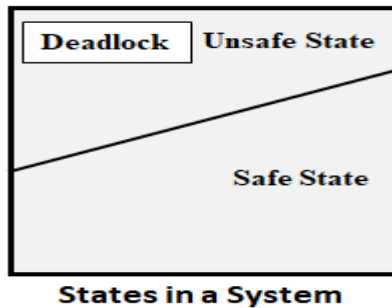


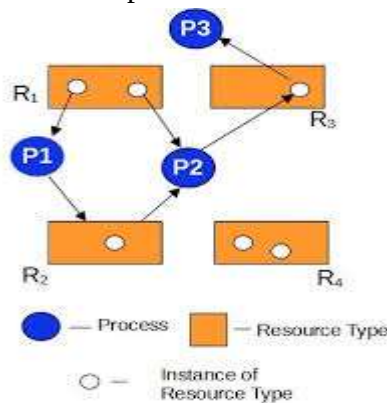
Figure: Safe, unsafe, and deadlock state spaces.

If no such sequence exists, then the system state is said to be unsafe.

2. Resource-Allocation Graph Algorithm

A Resource Allocation Graph (RAG) is a graphical way to represent the allocation of resources to processes in a system, helping to detect or avoid deadlocks.

- **Processes** – represented by circles (e.g., P1, P2).
- **Resources** – represented by squares (e.g., R1, R2).
- **Edges:**
 - **Request Edge ($\rightarrow$):** from a process to a resource (P1 $\rightarrow$ R1) – means the process is waiting for that resource.
 - **Assignment Edge ($\leftarrow$):** from a resource to a process (R1 $\leftarrow$ P1) – means the resource is currently assigned to that process.



Example: If P1 is holding R1 and waiting for R2, and P2 is holding R2 and waiting for R1, a cycle appears in the graph — indicating a deadlock.

3. Banker's Algorithm

It checks whether resource allocation is safe before granting a process's request.

The resource-allocation graph algorithm is not applicable to a resource allocation system with multiple instances of each resource type. The deadlock avoidance algorithm that we describe next is applicable to such a system, but is less efficient than the resource allocation graph scheme. This algorithm is commonly known as the banker's algorithm.

Key Data Structures:

1. **Available** – Resources currently available.
2. **Max** – Maximum demand of each process.
3. **Allocation** – Resources currently allocated to each process.
4. **Need** – Remaining resource need ($\text{Need} = \text{Max} - \text{Allocation}$).

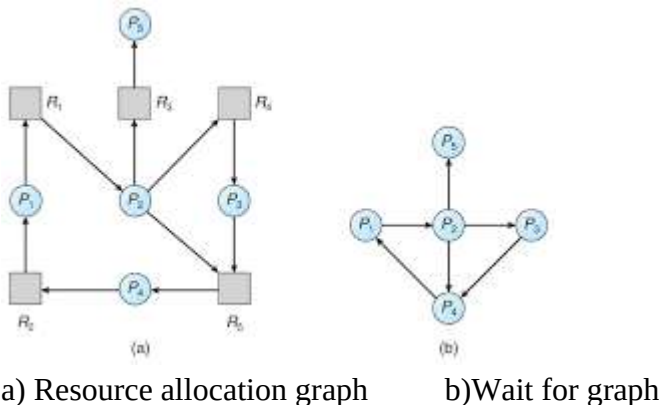
3) DEADLOCK DETECTION

If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur.

- An algorithm that examines the state of the system to determine whether a deadlock has occurred.
- An algorithm to recover from the deadlock.

Single Instance of Each Resource Type

If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a wait-for graph. We obtain this graph from the resource allocation graph by removing the nodes of type resource and collapsing the appropriate edges.

**Several Instances of a Resource Type**

The wait-for graph scheme is not applicable to a resource-allocation system with multiple instances of each resource type.

The algorithm used are :

- **Available:** [3, 2, 5] (3 printers, 2 monitors, 5 scanners are available)
- **Allocation:**
 - P1 has 1 printer and 1 monitor: [1, 1, 0]
 - P2 has 2 printers: [2, 0, 1]
- **Request:**
 - P1 wants 1 more printer and 1 more scanner: [1, 0, 1]
 - P2 wants 1 more monitor: [0, 1, 0]

This setup helps determine if a system can safely allocate resources without leading to deadlock.

4) RECOVERY FROM DEADLOCK

After detecting a deadlock, the system can either notify the operator for manual recovery or handle it automatically.

1. Process Termination

To eliminate deadlocks by aborting a process, we use one of two methods. In both methods, the system reclaims all resources allocated to the terminated processes.

- **Abort all deadlocked processes:** Stops all deadlocked processes, but loses all their work and may need to recompute.
- **Abort one process at a time:** Stops processes one by one until deadlock is broken, but takes more time due to repeated checks.

2. Resource Preemption

To eliminate deadlocks using resource preemption, we successively pre-empt some resources from processes and give these resources to other processes until the deadlock cycle is broken.

The three issues are considered to recover from deadlock.

1. Selecting a victim
2. Rollback
3. Starvation

- **Selecting a victim:** pick the process whose resources cost least to reclaim (e.g., smallest memory or CPU time).
- **Rollback:** ideally undo work to a safe checkpoint, but since that's hard, you often must restart the process from the beginning.
- **Starvation:** prevent a process from being preempted forever by boosting its priority each time it loses resources so it eventually runs to completion.